

# **Лабораторная работа № 3.**

## **Управляющие структуры**

Абакумова Олеся Максимовна, НФИбд-02-22

# Содержание

|          |                                                 |           |
|----------|-------------------------------------------------|-----------|
| <b>1</b> | <b>Цель работы</b>                              | <b>5</b>  |
| <b>2</b> | <b>Задание</b>                                  | <b>6</b>  |
| <b>3</b> | <b>Выполнение лабораторной работы</b>           | <b>7</b>  |
| 3.1      | Циклы while и for . . . . .                     | 7         |
| 3.2      | Условные выражения . . . . .                    | 9         |
| 3.3      | Функции . . . . .                               | 10        |
| 3.4      | Сторонние библиотеки (пакеты) в Julia . . . . . | 13        |
| 3.5      | Задания для самостоятельной работы . . . . .    | 14        |
| <b>4</b> | <b>Выводы</b>                                   | <b>23</b> |

## Список иллюстраций

|      |                                                                   |    |
|------|-------------------------------------------------------------------|----|
| 3.1  | Примеры с while и for . . . . .                                   | 8  |
| 3.2  | Использование цикла for для создание двумерного массива . . . . . | 9  |
| 3.3  | Пример условного выражения . . . . .                              | 10 |
| 3.4  | Пример написания функции . . . . .                                | 11 |
| 3.5  | Использование функций . . . . .                                   | 12 |
| 3.6  | Установка пакета, его подключение и пример с его использованием   | 13 |
| 3.7  | Задание 1 . . . . .                                               | 14 |
| 3.8  | Задание 1 . . . . .                                               | 15 |
| 3.9  | Задание 1, 2 и 3 . . . . .                                        | 16 |
| 3.10 | Задание 4, 5 и 6 . . . . .                                        | 17 |
| 3.11 | Задание 7 . . . . .                                               | 18 |
| 3.12 | Задание 7 . . . . .                                               | 19 |
| 3.13 | Задание 8 . . . . .                                               | 20 |
| 3.14 | Задание 9 . . . . .                                               | 21 |
| 3.15 | Задание 10 и 11 . . . . .                                         | 22 |

## **Список таблиц**

# 1 Цель работы

Основная цель работы — освоить применение циклов функций и сторонних для Julia пакетов для решения задач линейной алгебры и работы с матрицами.

## **2 Задание**

1. Выполнить примеры, приведенные в лабораторной работе.
2. Выполнить задания для самостоятельного выполнения

## 3 Выполнение лабораторной работы

### 3.1 Циклы while и for

Для различных операций, связанных с перебором индексируемых элементов структур данных, традиционно используются циклы while и for. Синтаксис while:

```
while <условие>  
    <тело цикла>  
end
```

Синтаксис for:

```
for <переменная> in <диапазон>  
    <тело цикла>  
end
```

Рассмотрим примеры (рис. 3.1):

### Циклы while и for

```
1]: # пока n<10 прибавить к n единицу и распечатать значение:  
n = 0  
while n < 10  
    n += 1  
    println(n)  
end
```

```
1  
2  
3  
4  
5  
6  
7  
8  
9  
10
```

```
2]: myfriends = ["Ted", "Robyn", "Barney", "Lily", "Marshall"]  
i = 1  
while i <= length(myfriends)  
    friend = myfriends[i]  
    println("Hi $friend, it's great to see you!")  
    i += 1  
end
```

```
Hi Ted, it's great to see you!  
Hi Robyn, it's great to see you!  
Hi Barney, it's great to see you!  
Hi Lily, it's great to see you!  
Hi Marshall, it's great to see you!
```

```
3]: for n in 1:2:10  
    println(n)  
end
```

```
1  
3  
5  
7  
9
```

```
4]: myfriends = ["Ted", "Robyn", "Barney", "Lily", "Marshall"]  
for friend in myfriends  
    println("Hi $friend, it's great to see you!")  
end
```

```
Hi Ted, it's great to see you!  
Hi Robyn, it's great to see you!  
Hi Barney, it's great to see you!  
Hi Lily, it's great to see you!  
Hi Marshall, it's great to see you!
```

```
5]: # инициализация массива m x n из нулей:  
m, n = 5, 5  
A = fill{0, (m, n)}
```

```
6]: 5x5 Matrix{Int64}:  
0 0 0 0 0  
0 0 0 0 0  
0 0 0 0 0  
0 0 0 0 0  
0 0 0 0 0
```

Рис. 3.1: Примеры с while и for

Пример использования цикла for для создания двумерного массива, в котором значение каждой записи является суммой индексов строки и столбца (рис. 3.2):



```

j): # формирование массива, в котором значение каждой записи
    # является суммой индексов строки и столбца:
    for i in 1:m
        for j in 1:n
            A[i, j] = i + j
        end
    end
end
A

j): 5x5 Matrix{Int64}:
 2 3 4 5 6
 3 4 5 6 7
 4 5 6 7 8
 5 6 7 8 9
 6 7 8 9 10

j): # инициализация массива m x n из нулей:
B = fill{0, (m, n)}

j): 5x5 Matrix{Int64}:
 0 0 0 0 0
 0 0 0 0 0
 0 0 0 0 0
 0 0 0 0 0
 0 0 0 0 0

j): for i in 1:m, j in 1:n
    B[i, j] = i + j
end
B

j): 5x5 Matrix{Int64}:
 2 3 4 5 6
 3 4 5 6 7
 4 5 6 7 8
 5 6 7 8 9
 6 7 8 9 10

j): C = [i + j for i in 1:m, j in 1:n]
C

j): 5x5 Matrix{Int64}:
 2 3 4 5 6
 3 4 5 6 7
 4 5 6 7 8
 5 6 7 8 9
 6 7 8 9 10

```

Рис. 3.2: Использование цикла for для создание двумерного массива

Также выше приведены другие различные реализации.

## 3.2 Условные выражения

Довольно часто при решении задач требуется проверить выполнение тех или иных условий. Для этого используют условные выражения. Синтаксис условных выражений с ключевым словом:

```

if <условие 1>
    <действие 1>
elseif <условие 2>
    <действие 2>
else
    <действие 3>
end

```

Рассмотрим пример (рис. 3.3):

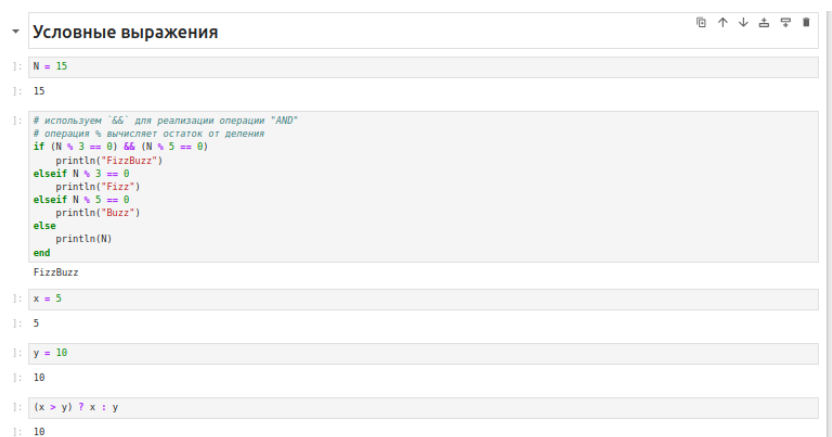


Рис. 3.3: Пример условного выражения

Синтаксис условных выражений с тернарными операторами:

$a ? b : c$

(если выполнено  $a$ , то выполнить  $b$ , если нет, то  $c$ ). Такая запись эквивалентна записи условного выражения с ключевым словом:

```

if a
    b
else
    c
end

```

Пример условного выражения с тернарным оператором также рассмотрен выше (рис. 3.3).

### 3.3 Функции

Julia дает нам несколько разных способов написать функцию. Первый требует ключевых слов `function` и `end` (рис. 3.4):

**Функции**

```

[30]: function sayhi(name)
      println("Hi $name, it's great to see you!")
      end

[30]: sayhi (generic function with 1 method)

[31]: # функция возведения в квадрат:
      function f(x)
          x^2
      end

[31]: f (generic function with 1 method)

[32]: sayhi("C-3P0")
      Hi C-3P0, it's great to see you!

[33]: f(42)
      1764

[34]: sayhi2(name) = println("Hi $name, it's great to see you!")
      sayhi2 (generic function with 1 method)

[35]: f2(x) = x^2
      f2 (generic function with 1 method)

[37]: sayhi3 = name -> println("Hi $name, it's great to see you!")
      #3 (generic function with 1 method)

[38]: f3 = x -> x^2
      #5 (generic function with 1 method)

[39]: # создаём массив v:
      v = [3, 5, 2]
      sort(v)
      v

[39]: 3-element Vector{Int64}:
       3
       5
       2

[40]: sort!(v)
      v

[40]: 3-element Vector{Int64}:
       2
       3
       5

[41]: map(f, [1, 2, 3])
      3-element Vector{Int64}:
       1
       4
       9

```

Рис. 3.4: Пример написания функции

По соглашению в Julia функции, сопровождаемые восклицательным знаком, изменяют свое содержимое, а функции без восклицательного знака не делают этого. Можно рассмотреть это на примере функции `sort`(рис. 3.4).

Функция `sort(v)` возвращает отсортированный массив, который содержит те же элементы, что и массив `v`, но исходный массив `v` остаётся без изменений. Если же использовать `sort!(v)`, то отсортировано будет содержимое исходного массива `v`.

В программировании под функцией высшего порядка понимается функция, принимающая в качестве аргументов другие функции или возвращающая другую функцию в качестве результата. Основная идея состоит в том, что функции имеют тот же статус, что и другие объекты данных.

- В Julia функция `map` (рис. 3.5) является функцией высшего порядка, которая принимает функцию в качестве одного из своих входных аргументов

и применяет эту функцию к каждому элементу структуры данных, которая ей передаётся также в качестве аргумента.

- Функция `broadcast` (рис. 3.5) — ещё одна функция высшего порядка в Julia, представляющая собой обобщение функции `map`. Функция `broadcast()` будет пытаться привести все объекты к общему измерению, `map()` будет напрямую применять данную функцию поэлементно.
- Точечный синтаксис (рис. 3.5) для `broadcast()` позволяет записать относительно сложные составные поэлементные выражения в форме, близкой к математической записи.

```
[42]: f(x) = x^2
      map(f, [1, 2, 3])

[42]: 3-element Vector{Int64}:
      1
      4
      9

[43]: x -> x^3
      map(x -> x^3, [1, 2, 3])

[43]: 3-element Vector{Int64}:
      1
      8
     27

[44]: f(x) = x^2
      broadcast(f, [1, 2, 3])

[44]: 3-element Vector{Int64}:
      1
      4
      9

[45]: f.([1, 2, 3])

[45]: 3-element Vector{Int64}:
      1
      4
      9

[46]: # Задаём матрицу A:
      A = [i + 3*j for j in 0:2, i in 1:3]

[46]: 3x3 Matrix{Int64}:
      1 2 3
      4 5 6
      7 8 9

[47]: f(A)

[47]: 3x3 Matrix{Int64}:
     30 36 42
     66 81 96
    102 126 150

[48]: B = f.(A)

[48]: 3x3 Matrix{Int64}:
      1 4 9
     16 25 36
     49 64 81

[49]: A .+ 2 .* f.(A) ./ A

[49]: 3x3 Matrix{Float64}:
      3.0 6.0 9.0
     12.0 15.0 18.0
     21.0 24.0 27.0

[50]: @. A + 2 * f(A) / A

[50]: 3x3 Matrix{Float64}:
      3.0 6.0 9.0
     12.0 15.0 18.0
     21.0 24.0 27.0

[51]: broadcast(x -> x + 2 * f(x) / x, A)

[51]: 3x3 Matrix{Float64}:
      3.0 6.0 9.0
     12.0 15.0 18.0
     21.0 24.0 27.0
```

Рис. 3.5: Использование функций

## 3.4 Сторонние библиотеки (пакеты) в Julia

Julia имеет более 2000 зарегистрированных пакетов, что делает их огромной частью экосистемы Julia. Есть вызовы функций первого класса для других языков, обеспечивающие интерфейсы сторонних функций. Можно вызвать функции из Python или R, например, с помощью PyCall или Rcall.

- При первом использовании пакета в текущей установке Julia вам необходимо использовать менеджер пакетов, чтобы явно его добавить (рис. 3.6).
- При каждом новом использовании Julia (например, в начале нового сеанса в REPL или открытии блокнота в первый раз) нужно загрузить пакет, используя ключевое слово `using` (рис. 3.6).
- Можно рассмотреть пример с использованием пакета `Colors` (рис. 3.6).

```
Сторонние библиотеки (пакеты) в Julia

[52]: import Pkg
[53]: Pkg.add("Example")

Updating registry at '~/.julia/registries/General.toml'
Resolving package versions...
Installed Example - v0.5.5
Updating '~/.julia/environments/v1.11/Project.toml'
[7876af07] + Example v0.5.5
Updating '~/.julia/environments/v1.11/Manifest.toml'
[7876af07] + Example v0.5.5
Precompiling project...
 5048.0 ms  ✓ Example
1 dependency successfully precompiled in 18 seconds. 650 already precompiled.

[54]: using Example
[55]: Pkg.add("Colors")

Resolving package versions...
No Changes to '~/.julia/environments/v1.11/Project.toml'
No Changes to '~/.julia/environments/v1.11/Manifest.toml'

[56]: using Colors
[57]: palette = distinguishable_colors(100)

[57]: 

[145]: rand(palette, 3, 3)

[145]: 
```

Рис. 3.6: Установка пакета, его подключение и пример с его использованием

## 3.5 Задания для самостоятельной работы

1. Используя циклы `while` и `for` (рис. 3.7):

- выведите на экран целые числа от 1 до 100 и напечатайте их квадраты;
- создайте словарь `squares`, который будет содержать целые числа в качестве ключей и квадраты в качестве их пар-значений;
- создайте массив `squares_arr`, содержащий квадраты всех чисел от 1 до 100.

```
Задания для самостоятельного выполнения

(5): using LinearAlgebra

# 1. Циклы и коллекции
println("1. Числа и их квадраты:")
# while
i = 1
while i <= 100
    println("$i -> $(i^2)")
    i += 1
end

1. Числа и их квадраты:
1 -> 1
2 -> 4
3 -> 9
4 -> 16
5 -> 25
6 -> 36
7 -> 49
8 -> 64
9 -> 81
10 -> 100
11 -> 121
12 -> 144
13 -> 169
14 -> 196
15 -> 225
16 -> 256
17 -> 289
18 -> 324
19 -> 361
20 -> 400
21 -> 441
22 -> 484
23 -> 529
24 -> 576
25 -> 625
26 -> 676
27 -> 729
28 -> 784
29 -> 841
30 -> 900
31 -> 961
32 -> 1024
33 -> 1089
34 -> 1156
35 -> 1225
36 -> 1296
37 -> 1369
38 -> 1444
39 -> 1521
40 -> 1600
41 -> 1681
42 -> 1764
```

Рис. 3.7: Задание 1

```
[106]: # for
      for i in 1:100
        println("$i -> $(i^2)")
      end

1 -> 1
2 -> 4
3 -> 9
4 -> 16
5 -> 25
6 -> 36
7 -> 49
8 -> 64
9 -> 81
10 -> 100
11 -> 121
12 -> 144
13 -> 169
14 -> 196
15 -> 225
16 -> 256
17 -> 289
18 -> 324
19 -> 361
20 -> 400
21 -> 441
22 -> 484
23 -> 529
24 -> 576
25 -> 625
26 -> 676
27 -> 729
28 -> 784
29 -> 841
30 -> 900
31 -> 961
32 -> 1024
33 -> 1089
34 -> 1156
35 -> 1225
36 -> 1296
37 -> 1369
38 -> 1444
39 -> 1521
40 -> 1600
41 -> 1681
42 -> 1764
43 -> 1849
44 -> 1936
45 -> 2025
46 -> 2116
47 -> 2209
48 -> 2304
49 -> 2401
50 -> 2500
51 -> 2601
52 -> 2704
```

Рис. 3.8: Задание 1

```

j): # Словарь
squares = Dict{Int, Int}()
for i in 1:100
    squares[i] = i^2
end

j): # Массив
squares_arr = [i^2 for i in 1:100]

j): 100-element Vector{Int64}:
 1
 4
 9
16
25
36
49
64
81
100
121
144
169
⋮
7921
8100
8281
8464
8649
8836
9025
9216
9409
9604
9801
10000

j): # 2. Проверка чётности
function check_parity(n)
    if n % 2 == 0
        println(n)
    else
        println("нечётное")
    end
end
check_parity(11)

нечётное

j): # Тернарный оператор
function check_parity_ternary(n)
    println(n % 2 == 0 ? n : "нечётное")
end
check_parity_ternary(5)

нечётное

j): x = 2
# 3. Функция add_one
add_one(x) = x + 1
add_one(x)

j): 3

```

Рис. 3.9: Задание 1, 2 и 3

2. Напишите условный оператор, который печатает число, если число чётное, и строку «нечётное», если число нечётное. Перепишите код, используя тернарный оператор (рис. 3.9).
3. Напишите функцию `add_one`, которая добавляет 1 к своему входу (рис. 3.9).
4. Используйте `map()` или `broadcast()` для задания матрицы `A`, каждый элемент которой увеличивается на единицу по сравнению с предыдущим (рис. 3.10).



```

74): # 4. Матрица с последовательными значениями
A = reshape(1:9, 3, 3) |> Matrix
A_plus_one = broadcast(x -> x + 1, A)

74): 3x3 Matrix{Int64}:
 2  5  8
 3  6  9
 4  7 10

73): # 5. Работа с матрицей A
A = [1 1 3; 5 2 6; -2 -1 -3]
k(x) = x^3
B = k.(A)

73): 3x3 Matrix{Int64}:
 1  1 27
125  8 216
 -8 -1 -27

74): # Замена третьего столбца
for i in 1:3
    A[i,3] = A[i,1] + A[i,2]
end
A

74): 3x3 Matrix{Int64}:
 1  1  2
 5  2  7
 -2 -1 -3

75): m, n = 2, 3
A = fill(1, (m, n))

75): 2x3 Matrix{Int64}:
 1  1  1
 1  1  1

77): # 6. Матрица B и произведение
B = [10 -10 10]
B = repeat(B, 15)
C = B' * B

77): 3x3 Matrix{Int64}:
1500 -1500 1500
-1500 1500 -1500
1500 -1500 1500

```

Рис. 3.10: Задание 4, 5 и 6

5. Задайте матрицу A и выполните (рис. 3.10):

- Возведите матрицу в третью степень.
- Замените третий столбец матрицы A на сумму второго и третьего столбцов.

6. Создайте матрицу B с элементами. Вычислите матрицу  $C = B^T B$  (рис. 3.10).

7. Создайте матрицу Z размерности 6 x 6, все элементы которой равны нулю, и матрицу E, все элементы которой равны 1. Используя циклы while или for и закономерности расположения элементов, создайте следующие матрицы размерности 6 x 6 (рис. 3.11).

```

76. # Задание 7 -|

# Создаем нулевую матрицу 6x6 и матрицу из единиц 6x6
Z = fill(0, (6, 6))

76]: 6x6 Matrix{Int64}:
 0 0 0 0 0 0
 0 0 0 0 0 0
 0 0 0 0 0 0
 0 0 0 0 0 0
 0 0 0 0 0 0
 0 0 0 0 0 0

77]: E = fill(1, (6, 6))

77]: 6x6 Matrix{Int64}:
 1 1 1 1 1 1
 1 1 1 1 1 1
 1 1 1 1 1 1
 1 1 1 1 1 1
 1 1 1 1 1 1
 1 1 1 1 1 1

90. # Создаем нулевые матрицы
Z1 = zeros(Int, 6, 6)
Z2 = zeros(Int, 6, 6)
Z3 = zeros(Int, 6, 6)
Z4 = zeros(Int, 6, 6)

90]: 6x6 Matrix{Int64}:
 0 0 0 0 0 0
 0 0 0 0 0 0
 0 0 0 0 0 0
 0 0 0 0 0 0
 0 0 0 0 0 0
 0 0 0 0 0 0

92]: # Z1 - единицы на побочных диагоналях (|i-j| == 1)
for i in 1:6, j in 1:6
    if abs(i - j) == 1
        Z1[i, j] = 1
    end
end
println("Z1:")
display(Z1)

Z1:
6x6 Matrix{Int64}:
 0 1 0 0 0 0
 1 0 1 0 0 0
 0 1 0 1 0 0
 0 0 1 0 1 0
 0 0 0 1 0 1
 0 0 0 0 1 0

93]: # Z2 - единицы на главной диагонали и через одну позицию (|i-j| == 0 или 2)
for i in 1:6, j in 1:6
    if i == j || abs(i - j) == 2
        Z2[i, j] = 1
    end
end
println("\nZ2:")
display(Z2)

Z2:
6x6 Matrix{Int64}:
 1 0 1 0 0 0
 0 1 0 1 0 0
 1 0 1 0 1 0
 0 1 0 1 0 1
 0 0 1 0 1 0
 0 0 0 1 0 1

```

Рис. 3.11: Задание 7

```

}4]: # Z3 - единицы в шахматном порядке с определенным паттерном
# Альтернативный подход с использованием индексов
Z3[1, [4, 6]] := 1
Z3[2, [3, 5]] := 1
Z3[3, [2, 4, 6]] := 1
Z3[4, [1, 3, 5]] := 1
Z3[5, [2, 4]] := 1
Z3[6, [1, 3]] := 1
println("\nZ3:")
display(Z3)

```

```

Z3:
6x6 Matrix{Int64}:
0 0 0 1 0 1
0 0 1 0 1 0
0 1 0 1 0 1
1 0 1 0 1 0
0 1 0 1 0 0
1 0 1 0 0 0

```

```

}5]: # Z4 - единицы на четных суммах индексов ((i+j) % 2 == 0)
for i in 1:6, j in 1:6
    if (i + j) % 2 == 0
        Z4[i, j] = 1
    end
end
println("\nZ4:")
display(Z4)

```

```

Z4:
6x6 Matrix{Int64}:
1 0 1 0 1 0
0 1 0 1 0 1
1 0 1 0 1 0
0 1 0 1 0 1
1 0 1 0 1 0
0 1 0 1 0 1

```

Рис. 3.12: Задание 7

8. В языке R есть функция `outer()`. Фактически, это матричное умножение с возможностью изменить применяемую операцию (например, заменить произведение на сложение или возведение в степень)/ Напишите эту функцию. (рис. 3.13).

```

22]: # Задание 8

# Функция outer выполняет операции между каждым элементом вектора x и y
# x .^ y' - возведение в степень (поэлементно)
# x .- y' - вычитание (поэлементно)
# x .% y' - взятие по модулю (поэлементно)
function outer(x, y, f)
    if (f == "**")
        res = x .^ y' # Возведение в степень
    elseif (f == "+")
        res = x .+ y'
    elseif (f == "^")
        res = x .^ y' # Возведение в степень (дублирование условия)
    elseif (f == "-")
        res = x .- y' # Вычитание
    elseif (f == "%")
        res = x .% y' # Взятие по модулю
    end
end

22]: outer (generic function with 1 method)

223.. # Создание матрицы 5x5 с операцией "+" ??????????????????
A1 = outer(0:4, 0:4, "+")
A1

23]: 5x5 Matrix{Int64}:
 0  1  2  3  4
 1  2  3  4  5
 2  3  4  5  6
 3  4  5  6  7
 4  5  6  7  8

78]: # Создание матрицы 5x5 с операцией "-."
A2 = outer(0:4, 1:5, "**")

78]: 5x5 Matrix{Int64}:
 0  0  0  0  0
 1  1  1  1  1
 2  4  8  16  32
 3  9  27  81  243
 4  16  64  256  1024

25]:
A3 = outer(0:4, 0:4, "+")
A3_res = outer(A3, 5, "%")

25]: 5x5 Matrix{Int64}:
 0  1  2  3  4
 1  2  3  4  0
 2  3  4  0  1
 3  4  0  1  2
 4  0  1  2  3

26]: A3 = outer(0:9, 0:9, "+")
A3_res = outer(A3, 10, "%")

26]: 10x10 Matrix{Int64}:
 0  1  2  3  4  5  6  7  8  9
 1  2  3  4  5  6  7  8  9  0
 2  3  4  5  6  7  8  9  0  1
 3  4  5  6  7  8  9  0  1  2
 4  5  6  7  8  9  0  1  2  3
 5  6  7  8  9  0  1  2  3  4
 6  7  8  9  0  1  2  3  4  5
 7  8  9  0  1  2  3  4  5  6
 8  9  0  1  2  3  4  5  6  7
 9  0  1  2  3  4  5  6  7  8

27]: A3 = outer(0:9, 0:9, "-.")
A3 = abs.(A3) # Берем абсолютные значения

27]: 10x10 Matrix{Int64}:
 0  1  2  3  4  5  6  7  8  9
 1  0  1  2  3  4  5  6  7  8
 2  1  0  1  2  3  4  5  6  7
 3  2  1  0  1  2  3  4  5  6
 4  3  2  1  0  1  2  3  4  5
 5  4  3  2  1  0  1  2  3  4
 6  5  4  3  2  1  0  1  2  3
 7  6  5  4  3  2  1  0  1  2
 8  7  6  5  4  3  2  1  0  1
 9  8  7  6  5  4  3  2  1  0

```

Рис. 3.13: Задание 8

9. Решите следующую систему линейных уравнений с 5 неизвестными (рис. 3.14):

$$\begin{cases} x_1 + 2x_2 + 3x_3 + 4x_4 + 5x_5 = 7, \\ 2x_1 + x_2 + 2x_3 + 3x_4 + 4x_5 = -1, \\ 3x_1 + 2x_2 + x_3 + 2x_4 + 3x_5 = -3, \\ 4x_1 + 3x_2 + 2x_3 + x_4 + 2x_5 = 5, \\ 5x_1 + 4x_2 + 3x_3 + 2x_4 + x_5 = 17, \end{cases}$$

рассмотрев соответствующее матричное уравнение  $Ax = y$ . Обратите внимание на особый вид матрицы  $A$ . Метод, используемый для решения данной системы уравнений, должен быть легко обобщаем на случай большего числа уравнений, где матрица  $A$  будет иметь такую же структуру.

```
# Задание 9

# Определяем матрицу коэффициентов A
A = [
    1 2 3 4 5;
    2 1 2 3 4;
    3 2 1 2 3;
    4 3 2 1 2;
    5 4 3 2 1
]

# Определяем вектор правой части y
y = [7; -1; -3; 5; 17]

# Решаем систему линейных уравнений A*x = y
x = A \ y

# Выводим решение системы
println("Решение системы: ")
println(x)

Решение системы:
[-2.0000000000000036, 3.0000000000000058, 4.999999999999998, 1.9999999999999991, -3.9999999999999999]
```

Рис. 3.14: Задание 9

10. Создайте матрицу  $M$  размерности  $6 \times 10$ , элементами которой являются целые числа (рис. 3.15).
11. Вычислите (рис. 3.15):

```

# Задание 10

# Создаем случайную матрицу 6x10 с числами от 1 до 10
M = rand(1:10, 6, 10)
println(M)

N = 4
K = 75

# Считаем сумму всех элементов матрицы
count_1 = sum(M, :N)
println(count_1)

# Идем строки, где сумма элементов равна 7
count_2 = [i for i=1:6 if sum(M[i,:]) == 7]
println(count_2)

# Идем пары столбцов, сумма которых больше K
count_3 = [(i, j) for i = 1:6, j = 2:5 if (i != j && sum(M[:, i] + M[:, j]) > K)]
println(count_3)

[7 6 10 3 10 10 5 5 3 8; 7 1 5 7 6 4 3 2 7 8; 10 4 4 4 1 2 2 7 3 3; 8 8 9 10 1 3 10 6 8 1; 7 5 8 7 5 5 2 9 6 4; 1 10 10 6 2 1 5 7 10 10]
38
[5]
[(3, 2), (1, 3), (2, 3), (4, 3), (1, 4), (3, 4)]

```

---

```

# Задание 11

# Вычисляем первую двойную сумму
# Для i от 1 до 20 и j от 1 до 5
# Формула: i^4 / (3 + j)
sum1 = sum(i^4 / (3 + j) for i in 1:20, j in 1:5)

# Вычисляем вторую двойную сумму
# Для i от 1 до 20 и j от 1 до 5
# Формула: i^4 / (3 + i * j)
sum2 = sum(i^4 / (3 + i * j) for i in 1:20, j in 1:5)

# Выводим результаты
println("Первая сумма: $sum1")
println("Вторая сумма: $sum2")

Первая сумма: 639215.2833333338
Вторая сумма: 89912.02146097131

```

Рис. 3.15: Задание 10 и 11

## 4 Выводы

В результате выполнения данной лабораторной работы я освоила применение циклов функций и сторонних для Julia пакетов для решения задач линейной алгебры и работы с матрицами.